CISC 322 Software Architecture W26

Conceptual Architecture of Gemini CLI

February 13, 2026

Authors:
Emanuel Caravaggio : 22gmx2@queensu.ca
Damon Clulow : 20djc9@queensu.ca
Raghab Dhungana : 21rd56@queensu.ca
Adam Elamin : 20aae5@queensu.ca
Paul Fernando : 20pf3@queensu.ca
Aditya Shetty : 21ass17@queensu.ca

Abstract

The Gemini CLI is an open-source tool that provides users a way to interact with Google's Gemini large language models from the command-line. The conceptual architecture involves a hybrid client-server style and uses a layered architectural approach that separates the concerns of the user interface from the concerns of model reasoning, tool execution, and system orchestration.

The CLI package, Core package, and Tool subsystem make up the Gemini CLI. The presentation layer is implemented in the CLI package and handles the user input and output. The application layer is implemented in the Core package, which takes user input and adds context to make prompts, manages state, and handles interactions with the Gemini API. The Tools subsystem is a suite of tools that provides the Gemini models with functionality to interact with the local environment through operations on the file system and shell commands. This module also provides interaction with external third-party services via the Model Context Protocol.

The report then outlines the behaviour of the Gemini CLI from both a control flow and data flow perspective. Control flow begins with system configuration, followed by request dispatch and model response handling, this flow continues in a loop until the session is finished. Data flow in the system is bidirectional and "V-shaped". To start, data flows upstream as user input is combined with context and constructed into a request, then processing is done by the Gemini model and data flows back downstream as tokens which are post-processed to be displayed to the user. Many of these steps occur asynchronously, ensuring that the user interface remains responsive while they are being completed.

Finally, the use cases of typical user interaction and automated issue triage are visualised. These help to demonstrate how the Gemini CLI has evolved over time from a tool to help developers to an agent that can perform independent software development work.

Table of Contents

Contents

Table of Contents 3
Introduction and Overview 4
Architecture 4
Components and Interactions 4
Architectural Styles 5
Control & Data Flow 6
Control Flow 6
Data Flow 6
Concurrency 7
External Interfaces 8
Use Cases 8
Use Case 1: The User Enters a Prompt in the Terminal 8
Interaction Summary 9
Use Case 2: Automated Issue Triage and Code Suggestion 9
Interaction Summary 10
System Evolution 10
General Growth 10
How the System Evolves – Maintainer/Developer Workflow 11
Naming Convention 11
Data Dictionary 12
Conclusion 12
Lessons Learnt 13
AI Collaboration Report 14
References 17

Introduction and Overview

The Gemini CLI is an open-source, cross-platform AI system that implements Google's Gemini, providing a flexible command line interface for interacting with the Gemini API. Created to be used as both a developer tool and user assistant, it facilitates seamless execution of commands, retrieval of resources, and integration with external systems, such as local file systems and development platforms like GitHub. Its architecture is layered, following a hybrid client-server style with clear separation between the frontend CLI package, backend Core package and Tools subsystem. This separation promotes maintainability and cross-platform reliability.

The architecture of Gemini CLI puts emphasis on modularity and layered responsibility. The CLI package acts as the presentation layer, which handles user input, displaying output as text or images and managing conversation history. The Core package serves as the application and logic layer, controlling operations such as session management, prompt construction and tool execution. It interfaces asynchronously with both the Gemini API and the Tools subsystem, which helps to extend the systems capabilities with file management, web operations, memory persistence and integration through Model Context Protocol (MCP). The layers communicate with one another through a defined control and data flow. Upstream flows transmit structured user input enriched with context to the Gemini API, then downstream flows deliver processed, human readable output back to the terminal. Concurrency is achieved via asynchronous execution, allowing the CLI package to remain interactive while the Core package handles API calls and tool operations.

Gemini CLI has evolved from a simple command execution tool to a powerful platform with its introduction of persistent memory, deep reasoning, and standardized extensions via MCP. developers contribute by a controlled Node.js workflow with automated preflight checks, continuous integration, and nightly releases. This allows the CLIs growth to be self-managing and maintained through automated processes. This evolutionary path guarantees Gemini CLI remains scalable, flexible, and capable of integrating new features while preserving a responsive, user focused experience across all operating systems.

Architecture

Components and Interactions

Google Gemini CLI is an open-source AI system that implements Google's Gemini large language models (LLMs) and functions. It functions as an AI assistant that can be launched from a web browser, a mobile app, or directly through the terminal. It executes operations within a localized environment, while accessing external file sources like the Google web browser and the user's file system. Gemini maintains conversation history across all chats to inform future interactions. The architecture follows a client–server style, divided into three main components, the CLI package, the Core package, and the Tools subsystem. The CLI is the user interface (UI) of the LLM. It takes care of user or terminal input, changing outputs into text or image representations, and handling UI customization and user-specific

configurations stored in ~/.gemini/settings.json. The Core package manages backend operations, it runs concurrently to the CLI, Gemini API, and Tools subsystem to decrease latency. It handles backend processing of prompts, state management, response processing, and general safety and security checks. Finally, the Tools subsystem extends previous services through use of many tools that interact with not only the local environment but external services as mentioned previously.

1. It can handle general file management using tools for reading, writing, searching, listing, and editing files.
2. It has shell tools for executing commands in a sandbox, so as not to affect user storage.
3. It uses web tools like Google searches and URL retrieval for online research.
4. Memory and persistence tools that store information across sessions and manage subtasks for complex requests.
5. Extension systems such as MCP bridges the API and Gemini, which can integrate systems like GitHub. These tools are registered by the Core package. The Core package makes sure that each tool is enabled and verified before use.

These components interact within a confined structure. Firstly, users input prompts that are registered by the CLI and analyzed by the Core package. The CLI then constructs prompts that use related conversation history. These can be adjusted or influenced by the user's configuration, what tools they have enabled and their authorization settings. The model then has options, it may return a direct response, request tool execution, or generate an image. Tools that are invoked, are factored into the output which is then processed and fed back into the model. The final response is delivered to the CLI and formatted appropriately. Conversation states are maintained by the Core package, with checkpointing and memory tools supporting long running chats that may require previous chatlog context to inform future decisions.

Architectural Styles

The Gemini CLI follows a hybrid architectural approach, as it utilizes Client-Server (Request-Response) architectural style, following a Layered Pattern. This allows the terminal side logic to be unaffected by the heavy processing that occurs on Google's infrastructure. We can view the different layers:

- **Presentation Layer**: Translates human intent into structured data and turn raw API responses into readable text.
    - **Key Responsibilities**: Input Handling, Prompt Preparation, Output Rendering
- **Application/Logic Layer**: Acts as the orchestrator in regard to the rules of the application.
    - **Key Responsibilities**: Session Management, Authentication Flow, Request Construction, Error Handling
- **Network/Service Layers**: Responsible for efficient communication between the CLI and remote Gemini services.

- **Key Responsibilities**: Protocol Management, Serialization, Asynchronous I/O, Connection Pooling
- **Server-side (Remote)**: Accounts for server-side responsibilities managed by Google's infrastructure.
  - **Key Responsibilities**: Inference Engine, Safety Layer, Load Balancing

Control & Data Flow

Control Flow

The control flow is outlined in Figure 1 it starts from when the user enters a command into the CLI, and follows a linear to remote processing, and back to the user though a structured pipeline. The logic for execution follows a clear sequence of stages: Initialization, Parsing, Execution, Termination. In the initialization stages. the CLI checks for configuration files and validates API credentials. This step ensures that the environment is properly set up, prior to processing any requests. Next, we move into the parsing stage, where the user inputs are decomposed into structured commands, flags, and arguments. This allows for parameters, command modifiers, and instructions to be interpreted correctly before execution, where the controller selects an appropriate action, and executes it. Lastly, in the termination stage, it will wait for a response, and the CLI either will terminate or wait for additional inputs in interactive mode.

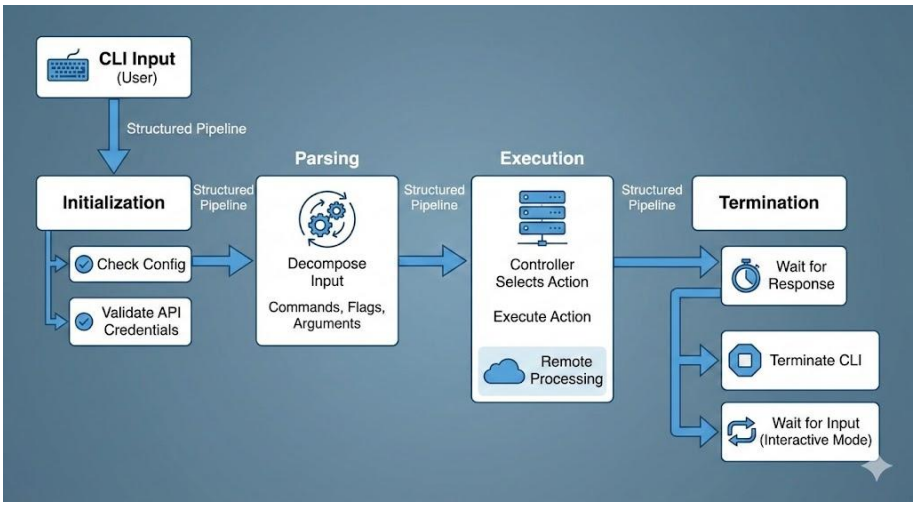

Figure 1: Figure showing the control flow through Gemini CLI

Data Flow

The data flow within the CLI is seen in Figure 2 and follows a bidirectional pipeline in which information originates at the client, undergoes significant transformation and enrichment, and is returned to the user through the terminal interface. This flow is often seen as a "V-shaped" data flow. We can refer to the process from the client to the server as the upstream flow, and the process from the server to the client as downstream flow.

Upstream Flow

The flow begins with the user's provided inputs. This input is then injected with context, and the CLI performs payload construction. This structured payload includes the user prompt, contextual data, and execution parameters. This payload is transmitted to the remote Gemini service for processing.

Downstream Flow

After the remote processing, Gemini's response is sent back. The response is sent as a stream of tokens. This response is then post-processed, applying formatting suitable for terminal display, and converting structured output into human-readable text. Then, the processed output is written to standard output, which are the presented results to the user.

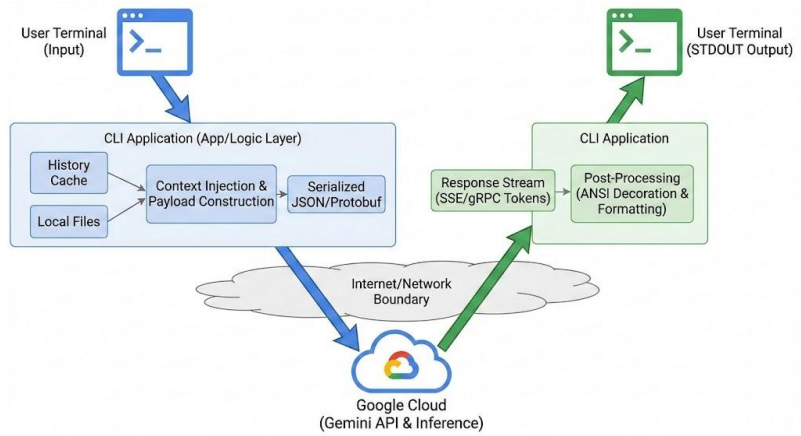

Figure 2: Diagram showing the data flow from user input to output being presented to the user

Concurrency

Although there is no mention of concurrency in the Gemini CLI documentation through exploring and experimenting with the system a concurrency model was found. Within the Gemini CLI concurrency is primarily asynchronous concurrency as the user interface remains responsive while the system waits on longer latency operations. This means that the CLI can continue providing output or receiving user input while the Core package is interacting with the Gemini API and performing tool execution. This allows the system to utilize the time that modules are waiting on external results to complete other work rather than relying on multithreading to complete multiple tasks at once.[1]

It is important to point out that this responsiveness does not mean parallel execution is happening as there is only a single thread of execution. This can be seen when entering a request while a previous request is still being processed, the input will be accepted but put on a queue until the previous request has been completed.[1]

External Interfaces

The Gemini CLI interacts with many external entities, including the user, the local operating system, remote cloud service, and more. The following interfaces facilitate the interactions.

User and Terminal Interface:

- The frontend for the LLM, processes user input and renders model responses into appropriate formats.
- Manages conversation history to maintain context across sessions.
- Handles personalized user preferences stored in **~.gemini/settings.json**.

File System and Shell Environment:

- Uses a suite of file system tools, to maintain integration with the host environment. Allows the model to perform read, write, batch-processing and other operations.
- It utilises **glob** and **grep** for discovering file patterns and searching content.

Model Context Protocol (MCP) and Extensions:

- Serves as the "universal connector" for the Gemini CLI.
- Allows the system to interface with third-party services without modifying the core codebase.
- Acts as the bridge for external platforms like **GitHub** for repository management, or **Slack** for team communications.
- The Extensions System allows developers to install pre-packaged integrations that bundle custom capabilities and specialized MCP server configs.

Use Cases

Use Case 1: The User Enters a Prompt in the Terminal

The sequence diagram in Figure 3 depicts the behavioural details of a user-initiated request within the Gemini CLI environment.

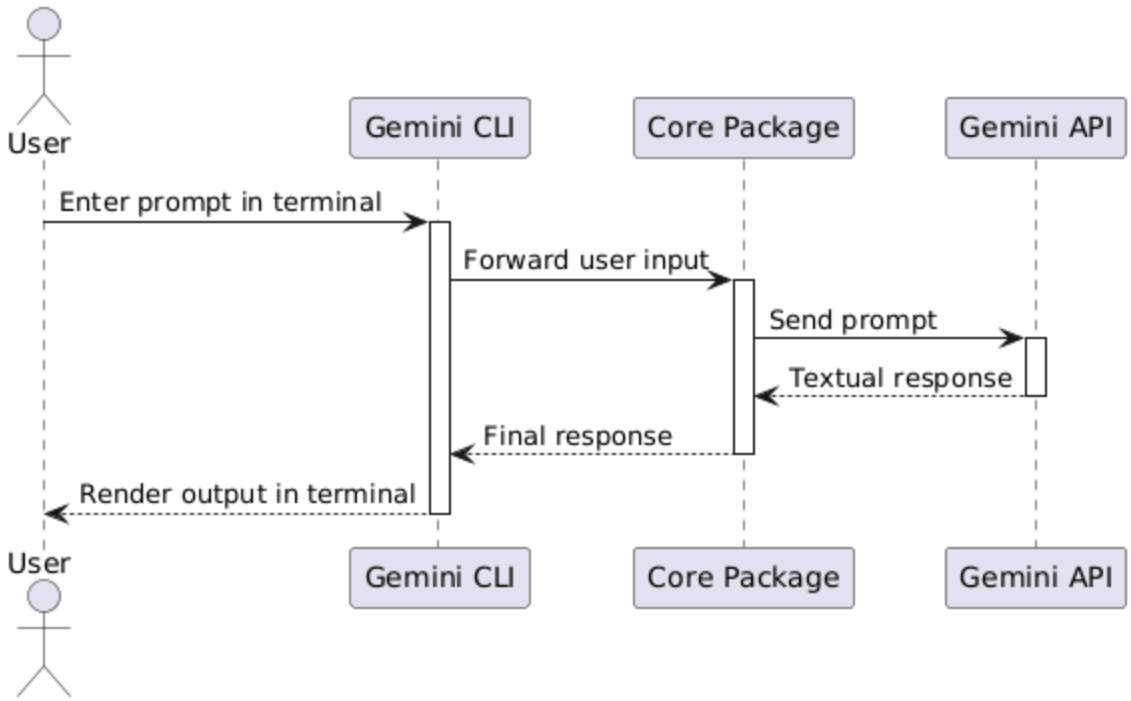

Figure 3: Sequence diagram depicting the behavioural details of a user-initiated request within the Gemini CLI environment.

Interaction Summary

- Initiation: The User (external actor) enters a prompt into the terminal to start the process.
- Input Processing: The Gemini CLI receives the entry and forwards the user input to the Core package.
- Request Dispatch: The Core package acts as the orchestrator, sending the prompt to the Gemini API for processing.
- API Response: The Gemini API returns a textual response back to the Core package.
- Result Delivery: The Core package passes the final response back to the Gemini CLI.
- Output Rendering: The Gemini CLI renders the output in the terminal for the user to view.

Use Case 2: Automated Issue Triage and Code Suggestion

This case visualized in Figure 4 demonstrates how the Gemini CLI operates as an automated agent within GitHub Actions to fetch repository context and provide issue triage.

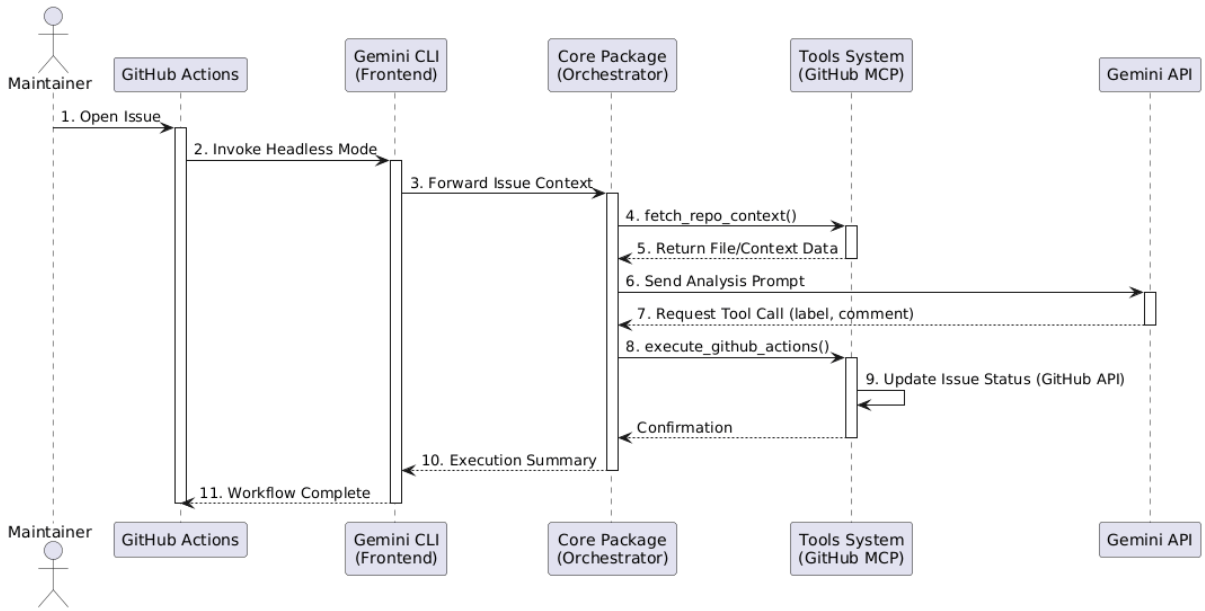

Figure 4: Sequence diagram depicting the behavioural details of the Gemini CLI operating as an autonomous agent within GitHub to triage issues.

Interaction Summary

- The process begins when the Maintainer (external actor) opens an issue, which GitHub Actions detects and uses to invoke the Gemini CLI.
- The Core package orchestrates the gathering of repository context, like existing file structures, using the Tools subsystem (GitHub MCP Tool).
- The Gemini API analyses issue context and tool schemas to decide on necessary actions, such as adding labels or suggesting code fixes.
- The Core package executes these decisions via the Tools subsystem autonomously.
- The system provides an execution summary and logs the completion back to the GitHub Actions environment.

System Evolution

General Growth

The Gemini CLI has grown from a simple developer tool into a complete system for independent software work by focusing on flexible design and user control. A major change happened with the addition of the MCP, a standard way for AI to connect to other programs. Often called the USB-C for AI, this protocol moved the tool away from fixed features toward a system where you can plug in new parts as needed. It acts as a bridge that lets the CLI find and use different data sources, like databases, Slack, or GitHub, through one standard setup.

This growth led to a design where the AI acts as an agent, using logical loops like Todo Planning to create a step-by-step plan for large requests. It also uses Sequential Approval, which is a safety check that keeps you in charge by asking for your okay before the AI makes sensitive changes. Newer versions use Gemini 3 Pro to add Deep Think skills and Persistent

Memory. These features let the CLI remember your project details and settings even after you close it.

How the System Evolves – Maintainer/Developer Workflow

Contributing developers follow a standardized Node.js workflow. The development environment asserts that developers must use Node.js version 20.19.0, to avoid dependency conflicts and mandates use of the **preflight** (>> npm run preflight) **script** which automates formatting, linting, and unit tests, before any pull request (PR) is submitted. [1]

Contribution lifecycle begins with forking the repository and implementing changes within the **packages/** directory, enforcing a division between the reasoning engine, the frontend, and specialized toolsets. Furthermore, all code contributions require a signed Contributor License Agreement (CLA). [2]

The systems evolution is "agentic", project maintainers utilise the Gemini CLI system itself to manage the repository's growth and maintenance. Automated GitHub actions, built using the CLI's frameworks, automates tasks such as issue triage, priority labelling, and code reviews. This dogfooding approach allows system evolution through self-correction. The project adheres to a continuous delivery model with nightly releases pushed at UTC 00:00, allowing developers to test the latest main branch features via the nightly tag. [3]

Naming Convention

- **Gemini CLI**: An open-source AI agent accessible from the command line that accepts user prompts and calls the Gemini model and can interact with the local environment to produce results depending on the prompt.
- **CLI Package:** Presentation layer responsible for the user-facing portion of the Gemini CLI.
- **Core Package:** System orchestrator responsible for constructing prompts, managing system state, executing tools and interacting with the Gemini.
- **Tool Subsystem:** Collection of tools enabling the model to interact with the local environment through file and shell operations and external third-party services.
- **Node.js:** An open-source JavaScript runtime environment that the Gemini CLI executes on. It provides the event loop as well as APIs for interacting with the environment.
- **Gemini API:** Remote service that is used to pass prompts to Google's Gemini models and receive responses.
- **Token:** Single unit of data that is processed by an AI model during inference. [4]
- **Model Context Protocol (MCP):** Open-source standard for AI applications to connect with third-party services. [5]
- **User Interface (UI):** The area of the program where the interactions between the user and program occurs.

- **GitHub:** Tool for storing and collaborating on software development projects hosted in the cloud.
- **Large Language Model (LLM):** Large deep learning models, able to take in natural language as input and produce natural language as output.

Data Dictionary

Tools - Verified tools that extend the capabilities of the Current Model

- **read_file** - Reads file
- **write_file** - Modifies file Content
- **list_directory** - Lists contents of a directory
- **run_shell_command** - Executes Shell Sandboxing
- **google_web_search** - Google Web Search
- **web_fetch** - Gets Web URLs
- **save_memory** - Memory retention across chat messages

Slash Commands - Commands that the User can Use to Interact with the CLI.

- **/help** - Help Information for Any Given Command
- **/chat** - Starts a new chat session
- **/memory** - Chat Memory Information
- **/bug** - Reports bugs or issues

Directory Structure - Organizes the Code Base into Distinct Components

- **/packages/cli/** - Frontend CLI/
- **packages/core/** - Backend For Computation Sequencing
- **/packages/core/src/tools/** - Modifiable Tool Interface

Environment Variables - Configuration on the System Level (Non-Client Specific) Logic

- **GEMINI_API_KEY** - API key for Gemini Authentication
- **GOOGLE_CLOUD_PROJECT -** Google Cloud Project Object Call Sequence

Conclusion

In conclusion, Gemini CLI is a strong and extensible open-source platform for interacting with Google's Gemini through a flexible and user focused command line interface. By implementing a hybrid architectural approach that combines client-server and layered styles, Gemini CLI achieves a strong balance between performance, maintainability, and scalability. This architecture allows the CLI package, Core package, and Tools subsystem to operate as integrated components while remaining distinct from one another.

Our analysis displays how Gemini CLI's core components collaborate with one another to provide essential functionality such as, session management, prompt organization, tool execution, and persistent memory. The well-defined control and data flows, supported by the system's asynchronous concurrency mechanisms, provides a responsive system even during costly operations like API calls and tool execution. Furthermore, the use of the MCP and extension system enables the CLI to connect to a diverse set of external services, remodelling from a simple assistant into a fully capable autonomous agent for software development and automation tasks.

By analysing Gemini CLI's conceptual architecture, use cases, and evolution, we have developed a conceptual model that highlights Gemini CLI's adaptability and long term growth. Its design, workflow and continuous delivery model allow for the system to evolve parallel to developer demands and advancements in AI technology. Overall, Gemini CLI is a robust, scalable framework that offers reliable and efficient environment building, automation, and workflow management across all platforms.

Lessons Learnt

Through the completion of this report, our team gained a deeper understanding of hybrid client-server systems. Furthermore, specific to the topic of the Gemini CLI, we learned about the necessity of separating presentation logic (i.e., the CLI package, the frontend, etc.) from the reasoning engine (Core/API). This separation defines the asynchronous concurrency (user can still type or interact to some degree with the UI, while the Core is waiting for the slow remote API to "think" or execute a tool); and the modularity (being able to update the underlying model logic or switch API versions in the Core without breaking the terminal) that was presented in this report about the system. However, this reliance on remote inference highlights a key bottleneck, and challenge of network latency, where system performance is directly dependent on the API availability.

From a collaboration standpoint, and the fact that the Gemini CLI is open source, the project reinforces the necessity of automated governance. Adherence to preflight checks, and continuous delivery illustrates the rigor needed to maintain stability in rapid development cycles.

Overall, the project considered the trade-offs in Gemini CLI's "agentic" design. While its modularity enables powerful automation and extensibility; reliance on the remote API and complex state management presents ongoing challenges. The experience emphasized the importance of a strong, thought-out, architectural foundation to support self-correcting and scalable development in modern AI applications.

AI Collaboration Report

Regarding our use of AI to aid us in our understanding of the conceptual architecture of Gemini CLI, we chose to use OpenAI's LLM ChatGPT - 5.2, February 2026 version, as well as Google's Gemini 3, November 2025 version. One reason we opted to include LLMs was for their ability to enhance our comprehension. When trying to determine the distinct components of a large system, like Gemini CLI, and how each component interacts to perform the system's functional requirements, the use of LLM's like the ones we chose can be beneficial. Due to the system's complexity, looking to understand the architecture by reading lines of code is not optimal. Instead LLMs can be used to read and analyze sections of code and system documentation, allowing easy understanding of the "big picture" of the system. Similar to writing or explaining code in pseudocode or even natural language, LLMs can provide a thorough understanding of the underlying architecture, helping us to identify any missing links or clarify any misconceptions. Similarly, another reason we opted to use these LLMs was for easy analysis and validation of architectural and design components of the Gemini CLI system and its external interfaces. These LLMs can be trained to look for specific patterns when provided proper supporting information. Using this information the models can help define and expose patterns being used within the system, helping to recognize how the system is following standard architectural styles that we see in all software systems. In addition to this, with the information given to it, the LLM can challenge our assumptions about the systems design. While these capabilities may not be specific to the models we chose, they are also popular, easily accessible, and generally known for being accurate, making them great additions to our team.

For our methodology towards interacting with the models, we allowed our entire group to collaborate in the action of crafting prompts and queries. Despite our ability to all interact with the LLM, we kept restrictions on the tasks we could give to the models we chose. These designated tasks include:

1. Reading system code and documentation, and providing a simplified rewording or explanation of the information given to it. We believed this was appropriate because the LLMs we decided to use are generally very good at reading and explaining code because of the training they do on massive open-source datasets.
2. Helping to verify assumptions made by group members, given that other members had been consulted prior and that the LLM has been given the necessary information needed to provide an accurate verification prior to being prompted.
3. Providing clarity or challenging the group when a member of the team is unsure about the function of a system component or an assumption being made about the system, given that other members had been consulted prior and that the LLM has been given the necessary information needed to provide an accurate verification prior to being prompted. While allowing it to be a tool of verification and clarification in reference to system architecture and component structure within a system is not LLMs strongest or most accurate function, we believed it was appropriate due to the many safety features we added onto the tasks. These include providing extensive information from

reliable sources, like our course content, and consulting other group members before using the LLM and about the LLM's output.

An example of a critical prompt we used is:

- Verified documentation was given to the LLM prior to this example prompt for training, including Gemini CLI documentation, "We are analyzing the conceptual architecture of Gemini CLI for a software architecture course. Our current architectural conclusion is that the system uses a hybrid client–server style combined with a layered architectural approach. Using the data given to you prior to this prompt please validate or challenge this conclusion with justification."

To further refine this prompt, we then reiterated constraints and specified our role to the LLM adding, "we our undergraduate students looking to gain an understanding of the conceptual architecture of the Gemini CLI system" and, "Only use the information provided below. Do not introduce external facts" to the prompt.

To ensure the output of our AI team member was void of factual errors, biases, or hallucinations we took precautions and actions to verify its contributions. The first step we took was ensuring all information provided to the LLMs that we thought would be necessary for query output, was fact-checked and valid itself. This ensured that if we asked the LLM to validate us on an assumption or to provide its own challenging opinion on an assumption the LLM would not provide inaccurate output containing factual error, simply from the information it is trained on. Upon receiving output from the LLM, the information we received would not be immediately taken as fact but rather read thoroughly. During this, we analyzed the LLM's output, using the documentation provided to us, in the form of course content and Gemini CLI documentation. We cross-referenced the output with these sources to ensure the LLM's output did not contain error, bias, or hallucinations, that could affect our understanding and presentation of the Gemini CLI conceptual architecture.

We believe that the LLMs we used played a less significant role, and were less substantial in the understanding of Gemini CLI's conceptual architecture than any one of our group members. For this reason, if you were to split the contribution evenly it would be just above 14% each, however; We think our use AI team member contributed around 10% of the final submitted deliverable. It was an impactful tool for simplifying documentation and in specific cases providing another form of validity to our system assumptions. Despite this, our understanding of the intricacies of Gemini CLI's architectural styles, component functions, component interactions, control and data flows and the systems evolution, was mainly from group conversation and individual understanding of the documentation provided to us. For these reasons we believe that our AI team members contributed 10% of our final deliverable, less than each of our individual group members.

Reflecting on our AI team members' effect on workflow, communication, and collaborative dynamics, we believe that although not the most optimal tool in some cases, overall, the AI team member made a positive effect on the group. Our AI member provided quick clarity to system documentation getting everybody up to speed on the system's "big picture", however; it also needed to be provided with considerable amounts of data to be used in more effective

ways, which became a setback in certain circumstances. It also played an effective role in settling group differences, leading us to stray away from disagreement, which was important under the time constraints of the deliverable. From using our AI member, our team learned that effective collaboration with AI comes not from letting AI make decisions for you, but instead, letting a well-prompted, informed, and cross-referenced AI, act as a form of validity to your own assumptions. This lesson we will continue to apply through each deliverable of this project and beyond the scope of this course.

References

[1] “Gemini CLI documentation,” Gemini CLI. Accessed: Feb. 09, 2026. [Online]. Available: https://geminicli.com/docs/
[2] “How to contribute,” Gemini CLI. Accessed: Feb. 09, 2026. [Online]. Available: https://geminicli.com/docs/contributing/
[3] “Automation and triage processes,” Gemini CLI. Accessed: Feb. 09, 2026. [Online]. Available: https://geminicli.com/docs/issue-and-pr-automation/
[4] D. Salvator, “Explaining Tokens — the Language and Currency of AI,” NVIDIA Blog. Accessed: Feb. 13, 2026. [Online]. Available: https://blogs.nvidia.com/blog/ai-tokens-explained/
[5] “What is the Model Context Protocol (MCP)?,” Model Context Protocol. Accessed: Feb. 13, 2026. [Online]. Available: https://modelcontextprotocol.io/docs/getting-started/intro